

Guia do projeto SEI com Hermes e LangGraph

Equipes, arquitetura técnica e preparação do estudo comparativo

CIIA • UnDF

Coordenação proposta André Rocha Vital

Planejamento de 21 de setembro a 10 de dezembro de 2026

Objetivo deste guia

Vamos construir dois agentes capazes de consultar documentos administrativos exportados do SEI e executar tarefas em um ambiente de testes. Um agente utilizará Hermes e o outro será implementado com LangGraph. Ambos compartilharão documentos, ferramentas e modelo de linguagem, permitindo comparar as implementações em condições documentadas.

Este guia orienta uma equipe de aproximadamente 15 pessoas, incluindo a coordenação. Explica o papel de cada frente, as tecnologias propostas, as interfaces entre componentes e as entregas necessárias para produzir um protótipo e um artigo até dezembro.

Resultado esperado

- Base documental organizada, pesquisável e vinculada aos processos de origem.
- Dois pipelines que consultam evidências, registram pendências e salvam minutas.
- Experimento reproduzível com resultados, análise de erros e manuscrito completo.

O que está definido e o que é proposta

A coleta manual dos documentos, a comparação Hermes–LangGraph e o prazo aproximado foram informados pela coordenação. A distribuição de pessoas, a stack, os formatos de API e as datas intermediárias abaixo são propostas de implementação. A versão e o repositório do Hermes, o modelo, a infraestrutura e o veículo do artigo precisam ser registrados pela equipe.

Os códigos usam dados fictícios e têm finalidade didática. Os trechos completos e os esqueletos de integração estão identificados; eles não constituem uma aplicação implantada nem uma integração com o SEI real.

1 O SEI e os documentos do projeto

SEI significa Sistema Eletrônico de Informações. Desenvolvido pelo Tribunal Regional Federal da 4ª Região, é um sistema de gestão de documentos e processos eletrônicos administrativos. Permite atividades como produção, edição, assinatura e tramitação documental. Integra as soluções do Processo Eletrônico Nacional. [1–3]

Um processo administrativo reúne documentos referentes a um assunto: uma solicitação, seus anexos, manifestações técnicas e decisões, por exemplo. O significado e os requisitos de cada procedimento devem ser confirmados com uma pessoa que conheça o domínio.

Conceito	Como aparece no projeto
Processo	Conjunto identificado de documentos relacionados a uma demanda.
Documento	Um ofício, requerimento, despacho, relatório ou anexo do processo.
Metadados	Identificador, tipo documental, data, unidade e outros campos disponíveis.
Minuta	Rascunho produzido para revisão; salvar a minuta não equivale a assinar ou tramitar.
Exportação	Cópia coletada em determinado momento; pode ter cobertura parcial.
Checklist	Requisitos da tarefa, com fonte, versão e validação do domínio.

Como usaremos o SEI

Os documentos já coletados manualmente formarão uma base própria de testes. A equipe preservará a relação com o processo e a página de origem. As ferramentas de escrita atuarão no banco e nos arquivos do protótipo. A inclusão de documentos, a assinatura e a tramitação no SEI institucional exigiriam uma etapa específica de integração e autorização operacional.

Caso de uso inicial

Pedido fictício: “Analise o processo PROC_001, confira os requisitos do procedimento e registre as pendências encontradas. Prepare uma minuta de complementação com as referências utilizadas.”

A saída esperada reúne a análise, as referências e os identificadores das ações gravadas. Se a exportação for parcial, o agente deve informar a limitação. Não localizar um documento na busca não comprova sua ausência no processo institucional.

2 Como as partes se conectam

A arquitetura separa a preparação documental, a execução de ferramentas e o controle do agente. O modelo interpreta o pedido e solicita operações; os adaptadores conduzem o ciclo; o backend valida e executa cada operação.

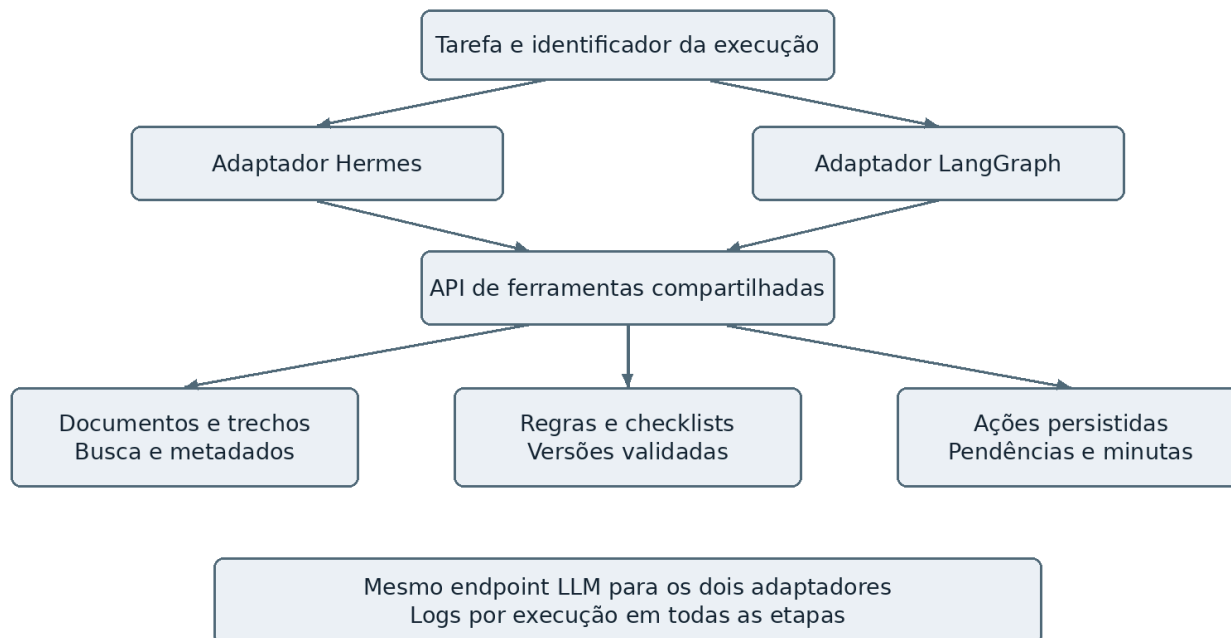


Figura 1 — Arquitetura proposta. As setas indicam solicitações; os resultados retornam pelo mesmo caminho. O endpoint LLM é consumido diretamente pelos adaptadores.

Uma tarefa do início ao fim

- A API registra o pedido e devolve um `run_id`, identificador único da execução.
- Um worker, processo que executa tarefas pendentes, seleciona o adaptador Hermes ou LangGraph.
- O agente consulta o modelo e aciona ferramentas da API compartilhada.
- As ferramentas leem documentos ou gravam ações e devolvem resultados estruturados.
- O agente usa os retornos para continuar ou finalizar; os registros permitem reconstruir a execução.

A ingestão dos PDFs acontece antes dessas tarefas. Reextrair e reindexar toda a base a cada pergunta aumentaria o tempo e alteraria a condição experimental.

3 Equipes e interfaces de trabalho

Frente	Pessoas	Entrega sob sua responsabilidade
Documentos	3	Base, extração, chunks, busca e conferência das fontes.
Backend	3	Banco, API, validação e ferramentas compartilhadas.
Hermes	2	Configuração, integração de ferramentas e logs do executor.
LangGraph	2	Grafo, estado, ferramentas e limites do executor.
Infraestrutura	2	VPS, containers, persistência e execução reproduzível.
Metodologia	2	Tarefas, rubricas, avaliação e análise experimental.
Coordenação	1	Arquitetura, prioridades, integração e edição do artigo.

Quem entrega o quê para quem

Documentos entrega ao backend textos, metadados e uma forma validada de recuperar trechos. Backend oferece ferramentas com entradas e saídas estáveis aos dois grupos de agentes. Infraestrutura disponibiliza o ambiente e a coleta de eventos. Metodologia especifica os campos dos registros e os critérios de sucesso; todos instrumentam seus componentes para atendê-los.

O mecanismo de busca tem responsabilidade compartilhada: documentos cuida da qualidade da recuperação; backend cuida de expô-la por API, aplicar filtros de acesso e padronizar os retornos. Uma alteração nessa interface precisa ser discutida pelos dois responsáveis.

Papel da coordenação

André acompanha as dependências entre frentes, revisa as decisões de arquitetura e consolida o manuscrito. Cada frente terá um responsável e cada entrega terá um revisor. O conhecimento deve circular por demonstrações, revisão de código e documentação curta.

Primeiro acordo entre equipes

Definir identificadores, versão da base, formatos de erro e os contratos de consultar_processo e registrar_pendencia. Contrato significa especificação da interface: quais campos entram, quais saem e o que acontece em caso de falha. Exemplos fictícios permitem avançar enquanto a base real é preparada.

4 Stack proposta e decisões de implementação

Componente	Stack proposta	Uso
Linguagem comum	Python 3.11 ou 3.12	Escolher uma versão compatível e fixá-la no projeto.
API e esquemas	FastAPI e Pydantic	Receber pedidos e validar campos de entrada e saída.
Banco	PostgreSQL e SQLAlchemy	Guardar metadados, ações e execuções; migrações com Alembic.
Extração	PyMuPDF	Extrair texto por página de PDFs com camada textual.
OCR quando necessário	Tesseract com português	Reconhecer texto em páginas digitalizadas.
Busca inicial	Filtros e busca textual PostgreSQL	Localizar por processo, documento e conteúdo.
Busca semântica opcional	Sentence Transformers e pgvector	Gerar embeddings e pesquisar vetores na mesma base.
Agentes	Hermes e LangGraph	Executores separados com capacidades equivalentes.
Cliente HTTP	HTTPX	Conectar adaptadores ao backend com timeout explícito.
Ambiente	Linux e Docker Compose	Executar serviços e volumes persistentes.
Análise e testes	pytest, pandas, SciPy e Matplotlib	Verificar comportamentos e analisar as rodadas.

Esta seleção é uma proposta, não um requisito de todas as bibliotecas. FastAPI usa modelos Pydantic para descrever corpos de requisição. PyMuPDF permite extração de texto; pgvector adiciona busca por similaridade ao PostgreSQL. [4–6]

Decisões que precisam ficar registradas

Escolher o repositório e commit do Hermes, versão do LangGraph, modelo e endpoint, modelo de embeddings e parâmetros de chunking. Fixar dependências em um arquivo de lock. Mudanças de modelo de embeddings exigem reindexação e devem gerar nova versão da base.

O MVP terá um modelo de geração e uma infraestrutura fixa. Redis, filas dedicadas, observabilidade externa, reranking e uma interface de chat elaborada podem ser incorporados se houver necessidade medida. O worker inicial pode consultar uma fila persistida no PostgreSQL, com mecanismo de reserva que evite executar a mesma tarefa simultaneamente.

5 Equipe de documentos

Essa frente transforma arquivos coletados em uma base que os agentes conseguem consultar e verificar. O produto é uma coleção rastreável, com qualidade de recuperação conhecida.

Pipeline de ingestão

- Inventariar arquivos e processos. Registrar origem, data da coleta, cobertura e hash do arquivo.
- Preparar os dados conforme o protocolo institucional, incluindo o tratamento de informações sensíveis previsto na metodologia.
- Extrair texto por página. Encaminhar páginas digitalizadas ou com extração ruim para OCR e revisão.
- Normalizar sem apagar informações relevantes, como números, datas, tabelas e referências.
- Dividir o texto respeitando seções e páginas. Preservar IDs e localização de cada trecho.
- Gerar embeddings se a busca semântica for adotada; armazenar modelo e versão usados.
- Conferir consultas conhecidas e documentar erros, lacunas e limitações.

Chunks e embeddings

Chunk é um trecho usado na recuperação. Embedding é uma representação numérica do conteúdo que permite comparar similaridade. A consulta e os trechos precisam ser representados pelo mesmo modelo compatível. Vetorizar documentos não treina nem atualiza os pesos do LLM que responde à tarefa.

Como ponto de partida para o piloto, experimentar trechos de 300–600 tokens com pequena sobreposição, ajustando pelo tokenizer e pelo limite do modelo escolhido. Esses valores são hipóteses de configuração; tabelas, títulos e referências podem exigir outro tratamento. Não cortar um requisito ao meio apenas para manter tamanho uniforme.

Critério de entrega

A frente demonstra dez consultas preparadas com a equipe de metodologia e aponta, para cada uma, o documento e a página que deveriam ser recuperados. A inspeção mede se a evidência correta está entre os resultados e se o texto foi extraído adequadamente.

Ao integrar a busca, o backend deve respeitar o filtro de processo antes de selecionar os resultados finais. A busca textual e o inventário continuam úteis para verificar presença documental; similaridade semântica isolada não é prova de completude.

6 Exemplo de extração e divisão do texto

Exemplo didático executável com PyMuPDF. A divisão abaixo usa caracteres para mostrar a mecânica e mantém cada trecho em sua página. No experimento, avaliar uma divisão por tokens ou estrutura documental. PDFs sem texto são sinalizados para OCR. [5]

```
import hashlib
import pymupdf

def dividir(texto, tamanho=1400, sobreposicao=200):
    if not 0 <= sobreposicao < tamanho:
        raise ValueError("Sobreposicao invalida")
    inicio = 0
    while inicio < len(texto):
        fim = min(inicio + tamanho, len(texto))
        yield texto[inicio:fim]
        if fim == len(texto):
            break
        inicio = fim - sobreposicao

def extrair(caminho, processo_id, documento_id):
    arquivo = open(caminho, "rb").read()
    versao = hashlib.sha256(arquivo).hexdigest()
    trechos, revisar = [], []
    with pymupdf.open(caminho) as pdf:
        for pagina, folha in enumerate(pdf, start=1):
            texto = folha.get_text("text").strip()
            if not texto:
                revisar.append(pagina)
                continue
            for ordem, trecho in enumerate(dividir(texto)):
                trechos.append({
                    "processo_id": processo_id,
                    "documento_id": documento_id,
                    "pagina": pagina,
                    "ordem": ordem,
                    "texto": trecho,
                    "arquivo_sha256": versao,
                })
    return {"trechos": trechos, "revisar_ocr": revisar}
```

Como evoluir o exemplo

Persistir os trechos no banco com IDs estáveis por versão; registrar falhas; conferir extrações aparentemente válidas; processar OCR de forma separada. Um PDF pode ter texto extraído incorretamente mesmo sem estar vazio. A amostra de conferência precisa incluir tabelas e páginas com layouts diferentes.

Se houver vetorização, gerar embeddings em lotes para os trechos aprovados e indexar somente essa versão. Registrar dimensão, modelo, normalização e métrica de similaridade. O mesmo procedimento deve atender os dois agentes.

7 Equipe de backend e catálogo de ferramentas

Backend implementa funções confiáveis e as expõe por API. A equipe valida dados, controla acesso, consulta a base e persiste ações. O modelo solicita uma operação; quem a executa é o código da ferramenta.

Ferramenta	Entrada principal	Resultado
consultar_processo	processo_id	Metadados, inventário e cobertura da exportação.
buscar_documentos	consulta, processo_id, filtros	Trechos, documentos, páginas e versão da base.
ler_documento	documento_id, páginas	Texto e referências da fonte solicitada.
consultar_requisitos	tipo_procedimento	Checklist validado, versão e fonte da regra.
registrar_pendencia	processo, requisito, descrição, evidências	ID da pendência e confirmação da gravação.
salvar_minuta	processo, conteúdo, referências	ID da minuta persistida e referência de acesso.

O que cada ferramenta precisa garantir

Toda ferramenta tem nome, descrição, esquema de entrada, esquema de saída e erros definidos. Leituras preservam a rastreabilidade. Escritas validam o processo, referências e campos obrigatórios; usam transações e uma chave de idempotência para evitar duplicação em uma repetição da mesma operação.

O identificador da execução deve vir do contexto autenticado do worker. O modelo não escolhe qual usuário está acessando a base nem pode ampliar sua própria permissão informando um ID diferente.

Interfaces propostas

Rota	Função
POST /runs	Criar tarefa e retornar run_id para acompanhamento.
GET /runs/{run_id}	Consultar andamento e resultado.
GET /processos/{processo_id}	Consultar metadados e inventário.
POST /buscas	Pesquisar conteúdo com filtros.
POST /pendencias	Registrar pendência no ambiente de testes.
POST /minutas	Salvar minuta e suas referências.

HTTP 404 indica recurso inexistente; 422 indica campos inválidos; 409 pode indicar conflito de idempotência. Falhas de infraestrutura precisam ser distinguíveis de “nenhum documento encontrado”.

8 Exemplo de API e validação de entrada

Os códigos das seções 8 e 9 formam um único arquivo `app.py`. O exemplo usa memória e dados fictícios para demonstrar o contrato. Ao reiniciar, as pendências desaparecem. Ele não inclui autenticação nem persistência e deve ser usado apenas em desenvolvimento local. [4]

```
import hashlib
import json
from uuid import uuid4
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI()
processos = {
    "PROC_001": {
        "tipo": "procedimento_demonstracao",
        "documentos": ["DOC_001"],
        "cobertura": "parcial",
    }
}
pendencias = {}
idempotencia = {}

class PendenciaEntrada(BaseModel):
    processo_id: str
    requisito_id: str
    descricao: str = Field(min_length=10, max_length=2000)
    evidencias: list[str] = Field(min_length=1)
    chave_idempotencia: str = Field(min_length=8)

@app.get("/processos/{processo_id}")
def consultar_processo(processo_id: str):
    if processo_id not in processos:
        raise HTTPException(404, "Processo inexistente")
    return {"processo_id": processo_id,
            **processos[processo_id]}
```

O que a equipe deve observar

`BaseModel` descreve o corpo da requisição. `Field` define restrições de tamanho e obrigatoriedade. O endpoint consulta um identificador conhecido e devolve um objeto JSON. A documentação do FastAPI explica como os modelos são utilizados para representar e validar os dados recebidos. [4]

Os identificadores de evidência precisam ser resolvidos no servidor. Para ausência documental, a evidência pode ser uma referência ao inventário e à cobertura da exportação; não se deve inventar uma página de um documento ausente.

9 Exemplo de ferramenta de escrita

Continuação do app.py. A chave de idempotência identifica uma operação lógica. Repetir a mesma chave e os mesmos dados devolve o registro anterior; reutilizar a chave com outro conteúdo produz conflito.

```
@app.post("/pendencias")
def registrar_pendencia(entrada: PendenciaEntrada):
    processo = processos.get(entrada.processo_id)
    if processo is None:
        raise HTTPException(404, "Processo inexistente")

    permitidas = set(processo["documentos"])
    permitidas.add("inventario:" + entrada.processo_id)
    if not set(entrada.evidencias) <= permitidas:
        raise HTTPException(422, "Evidencia desconhecida")

    dados = entrada.model_dump(
        exclude={"chave_idempotencia"}
    )
    resumo = hashlib.sha256(
        json.dumps(dados, sort_keys=True).encode()
    ).hexdigest()
    chave = entrada.chave_idempotencia
    if chave in idempotencia:
        anterior, registro_id = idempotencia[chave]
        if anterior != resumo:
            raise HTTPException(409, "Chave em conflito")
        return pendencias[registro_id]

    registro_id = str(uuid4())
    registro = {"pendencia_id": registro_id,
               "status": "registrada", **dados}
    pendencias[registro_id] = registro
    idempotencia[chave] = (resumo, registro_id)
    return registro
```

Para executar após instalar FastAPI e Uvicorn: `python -m uvicorn app:app --host 127.0.0.1 --port 8000`. A documentação interativa fica em <http://127.0.0.1:8000/docs>.

Evolução necessária para o experimento

Substituir dicionários por PostgreSQL, usar restrição única e transação para a idempotência, validar requisito e versão, associar a execução autenticada e definir modelo de resposta. A implementação em memória acima não é segura para múltiplos workers concorrentes.

A validação de IDs garante que a referência existe; a avaliação semântica verifica se ela realmente sustenta a pendência. Essas são verificações diferentes e ambas importam.

10 Como as ferramentas chegam aos agentes

Uma camada pequena traduz a chamada do agente em HTTP. O backend preserva a lógica de negócio. O exemplo abaixo cria duas funções ligadas ao contexto da execução; as duas podem ser utilizadas pelos adaptadores. O `run_id` é fornecido pelo executor.

```
import httpx

def criar_funcoes(base_url, run_id):
    async def consultar_processo(processo_id: str) -> dict:
        """Consulta metadados e inventario de um processo."""
        async with httpx.AsyncClient(
            base_url=base_url, timeout=20.0
        ) as cliente:
            r = await cliente.get(
                f"/processos/{processo_id}"
            )
            r.raise_for_status()
            return r.json()

    async def registrar_pendencia(
        processo_id: str, requisito_id: str,
        descricao: str, evidencias: list[str]
    ) -> dict:
        """Registra uma pendencia sustentada por evidencias."""
        dados = {
            "processo_id": processo_id,
            "requisito_id": requisito_id,
            "descricao": descricao,
            "evidencias": evidencias,
            "chave_idempotencia":
                f"{run_id}:{processo_id}:{requisito_id}",
        }
        async with httpx.AsyncClient(
            base_url=base_url, timeout=20.0
        ) as cliente:
            r = await cliente.post("/pendencias", json=dados)
            r.raise_for_status()
            return r.json()

    return [consultar_processo, registrar_pendencia]
```

Essa chave pressupõe uma pendência por requisito, processo e execução. Se o domínio permitir várias, incluir um identificador de ocorrência estável. Em produção, reutilizar o cliente HTTP e adicionar autenticação, correlação e tradução de erros. Timeout não significa que uma escrita falhou: a confirmação pode ter se perdido após a gravação.

11 Equipe Hermes e aplicação proposta

Hermes designa aqui o executor mencionado na metodologia do projeto. Seu repositório e sua versão ainda não foram fornecidos. Este capítulo define a integração desejada sem atribuir ao software APIs, classes, memória ou suporte a MCP que ainda não foram verificados.

O papel do Hermes no projeto

O executor recebe o pedido, prepara mensagens e definições das ferramentas, chama o LLM, encaminha suas solicitações ao backend e incorpora os retornos ao contexto. A sessão continua até uma resposta final, um impedimento ou um limite de execução. O LLM interpreta e escolhe operações; o runtime executa as chamadas.

Trabalho da equipe

- Identificar repositório, licença, commit e forma de inicialização. Registrar como o modelo é configurado.
- Localizar o mecanismo de ferramentas personalizadas. Verificar suporte a esquemas, chamadas assíncronas e coleta de eventos.
- Integrar as seis ferramentas administrativas à API compartilhada.
- Configurar instruções de evidência, limites e confirmação de escrita.
- Isolar sessões e controlar memória, habilidades aprendidas e outras capacidades que possam mudar entre tarefas.
- Exportar entradas, chamadas, retornos, resultado e consumo disponível no formato comum.

Prompts e ferramentas internas

O prompt deve instruir o agente a consultar requisitos, preservar referências e declarar limites da exportação. Uma persona pode orientar o tom; ela não substitui ferramentas funcionais nem validação. Ferramentas internas de terminal, arquivos ou navegador, se existirem, não devem permitir contornar a API no experimento principal. As capacidades habilitadas precisam ser equivalentes às do outro executor.

Esqueleto conceitual do adaptador

```
# Pseudocódigo: nomes abaixo não são uma API do Hermes.
iniciar_sessao_isolada(run_id)
configurar_modelo(modelo_comum)
registrar_ferramentas(catalogo_compartilhado)
enviar_tarefa(pedido, regras_comuns)
enquanto houver_execucao_e_limite_disponivel:
    evento = receber_evento_do_runtime()
    registrar_evento(evento)
    se evento_solicitar_tool:
        devolver_resultado(chamar_api_compartilhada(evento))
salvar_resultado_e_encerrar_sessao()
```

Critério de entrega: a mesma tarefa de consulta e gravação usada no LangGraph conclui com retorno confirmado pelo backend e histórico completo. A primeira integração usa dados fictícios.

12 Equipe LangGraph e exemplo de grafo

LangGraph é um framework de orquestração com estado. Neste desenho, um nó chama o modelo e outro executa ferramentas. Uma condição decide se o ciclo continua. LangChain fornece componentes úteis, mas LangGraph pode ser usado sem ele. [7–8]

Esqueleto de integração: a função recebe um modelo já configurado e funções como as da seção 10. Requer langgraph e langchain-core. O provedor precisa suportar bind_tools e chamadas assíncronas; essa configuração deve ser validada no piloto.

```
from langchain_core.tools import tool
from langchain_core.messages import SystemMessage, HumanMessage
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode

def construir_grafo(llm, funcoes):
    tools = [tool(funcao) for funcao in funcoes]
    modelo = llm.bind_tools(tools)

    async def agente(state):
        regras = SystemMessage(content=(
            "Use evidencias da base. Confirme escritas "
            "pelo retorno da ferramenta. Informe limitacoes."
        ))
        resposta = await modelo.ainvoke(
            [regras, *state["messages"]]
        )
        return {"messages": [resposta]}

    def proximo(state):
        ultima = state["messages"][-1]
        return "tools" if ultima.tool_calls else END

    fluxo = StateGraph(MessagesState)
    fluxo.add_node("agente", agente)
    fluxo.add_node("tools", ToolNode(tools))
    fluxo.add_edge(START, "agente")
    fluxo.add_conditional_edges(
        "agente", proximo, {"tools": "tools", END: END}
    )
    fluxo.add_edge("tools", "agente")
    return fluxo.compile()

async def executar(grafo, pedido):
    return await grafo.ainvoke(
        {"messages": [HumanMessage(content=pedido)]},
        config={"recursion_limit": 20},
    )
```

O limite de recursão é um limite de passos do grafo, não uma contagem equivalente de chamadas ao LLM. O runner deve impor também limites comuns de tempo, chamadas e consumo. A política de erros deve ser configurada explicitamente após fixar a versão das bibliotecas.

13 Estado persistência e tratamento de erros

O estado reúne informações da execução. No exemplo, MessagesState acumula mensagens e respostas de ferramentas. Os campos operacionais, como run_id e versão da base, podem ficar em um estado próprio ou no contexto do runner. A equipe deve definir uma única convenção para ambos os adaptadores.

Persistência durante a tarefa

Checkpoints permitem salvar e retomar o estado do LangGraph. Para o MVP, podem ser adicionados após o ciclo básico funcionar. Cada tarefa do benchmark deve usar uma identificação isolada; retomar uma tarefa interrompida é diferente de reutilizar memória entre tarefas. [9]

Situação	Comportamento esperado
Processo inexistente	Retornar erro de recurso; informar impedimento sem inventar conteúdo.
Busca sem resultado	Distinguir ausência de correspondência de falha de busca.
Exportação parcial	Limitar a conclusão à base disponível.
Argumento inválido	Devolver erro estruturado; permitir correção dentro dos limites.
Timeout em escrita	Consultar ou repetir com a mesma chave de idempotência.
Erro temporário do servidor	Aplicar política de tentativas limitada e registrar cada tentativa.
Limite de execução atingido	Encerrar com status específico e preservar os registros.

O que não entra no prompt

Credenciais, decisões de autorização e o gabarito do teste não devem entrar no contexto do modelo. O backend aplica as permissões com base no contexto autenticado. Conteúdo encontrado nos PDFs é evidência a analisar, não instrução para mudar regras ou acessar recursos adicionais.

Critério de entrega das duas frentes de agentes

Demonstrar uma tarefa concluída, uma tarefa impedida por falta de informação e uma recuperação de falha temporária. Mostrar os eventos que sustentam cada resultado. A resposta final só pode afirmar uma gravação quando houver confirmação, e não apenas porque o modelo produziu o texto correspondente.

14 Equipe de infraestrutura e hospedagem

Uma VPS é um servidor virtual com recursos de CPU, memória, disco e rede. É onde os serviços podem permanecer em execução. Hostinger é uma opção de provedor; seus produtos de VPS oferecem implantação com Docker. O plano e os recursos devem ser escolhidos após medir o piloto. [10]

Aplicação e inferência

API, banco e workers podem ficar na VPS. O modelo de linguagem pode estar em outro servidor com GPU, ligado por endpoint autenticado, ou em uma API autorizada para os dados. Hospedar documentos na VPS não implica executar o modelo nela; os trechos enviados ao endpoint também fazem parte do fluxo de dados.

Serviço	Onde fica na proposta	O que precisa
API de ferramentas	VPS	Rede interna, validação, autenticação e healthcheck.
PostgreSQL	VPS	Volume persistente, backups e migrações.
Arquivos	Volume persistente	Originais, textos extraídos e minutas versionadas.
Workers dos agentes	VPS	Credenciais, limites, acesso à API e ao LLM.
Endpoint LLM	GPU institucional ou serviço autorizado	Modelo fixado e suporte testado a tool calling.
Entrada HTTPS	Proxy na VPS	Domínio, certificado e acesso controlado.
Logs e resultados	Banco e arquivos do projeto	Correlação por run_id e retenção definida.

Responsabilidades da frente

- Preparar Linux, acesso SSH e configuração dos serviços.
- Criar Dockerfiles e Compose; fixar imagens para as rodadas finais.
- Definir volumes, backup e restauração testada dos dados.
- Disponibilizar configurações sem colocar segredos no Git.
- Executar o runner e preservar logs, artefatos e versões de cada rodada.

No desenvolvimento, a API pode ficar acessível apenas em localhost. Na implantação, banco e ferramentas internas permanecem na rede dos serviços. Um endpoint externo deve ter autenticação e HTTPS. Recursos e custos de hospedagem ainda são estimativas a levantar; não há contratação prevista neste guia.

15 Exemplo de implantação com Docker Compose

Esqueleto de infraestrutura. Requer um diretório backend com Dockerfile e uma aplicação que use DATABASE_URL e FILES_DIR. O app.py didático das páginas anteriores ainda não consome essas variáveis. O exemplo não inclui worker, proxy HTTPS ou autenticação; esses componentes entram na integração. [11]

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: sei_bench
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${DB_PASSWORD:?defina DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d sei_bench"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    build: ./backend
    environment:
      DATABASE_URL: ${DATABASE_URL:?defina DATABASE_URL}
      FILES_DIR: /data
    ports:
      - "127.0.0.1:8000:8000"
    volumes:
      - arquivos:/data
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

volumes:
  pgdata:
  arquivos:
```

DATABASE_URL deve apontar para o host db dentro da rede do Compose. Caracteres especiais da senha precisam ser codificados corretamente na URL. Workers em containers acessam http://api:8000; programas fora dos containers acessam a porta publicada. localhost dentro de um container aponta para ele próprio.

O serviço postgres:16 acima não inclui pgvector. Se a busca vetorial for adotada, a equipe deve usar uma imagem testada com a extensão, fixar sua versão e executar a migração correspondente. Os volumes preservam dados entre reinícios; backup é uma cópia independente que permite restauração.

16 Metodologia e desenho do experimento

A frente de metodologia começa junto com o desenvolvimento. Ela define tarefas, critérios de sucesso e gabaritos com apoio de alguém que conheça os processos. Também revisa se a implementação mede a pergunta de pesquisa pretendida.

Pergunta proposta

Sob o mesmo modelo, ferramentas e base documental, como implementações com Hermes e LangGraph diferem na conclusão de tarefas administrativas, no uso de ferramentas, na latência e no custo?

Os resultados se referem às versões e configurações testadas. Comparar dois executores não permite concluir, por si só, qual é o efeito de usar qualquer orquestrador em relação a um modelo sem esse suporte. Um controle separado seria necessário para essa outra pergunta.

Decisão	Proposta inicial
Famílias	Consulta e síntese; conferência documental; minuta com ação registrada.
Desenvolvimento	12–15 tarefas abertas para ajuste e piloto.
Teste final	Meta de 40–60 tarefas reservadas, definidas antes da coleta.
Separação	Separar por processo ou conjunto documental para reduzir vazamento.
Repetições	Três por tarefa e executor; contexto e estado inicial isolados.
Modelo e ambiente	Um modelo e uma infraestrutura fixos na comparação principal.
Quantidade ilustrativa	60 tarefas × 2 executores × 3 repetições = 360 execuções.

O tamanho da amostra acima é uma proposta operacional. Definir a análise e verificar sua adequação com o orientador antes da coleta final. Repetições da mesma tarefa e tarefas do mesmo processo podem ser dependentes; não tratá-las automaticamente como observações independentes. A escolha do teste deve considerar o desenho e a métrica. [12]

Equivalência entre condições

Manter base, busca, regras, ferramentas, modelo, instruções de tarefa e orçamento de execução equivalentes. Restaurar as escritas entre rodadas e alternar ou randomizar a ordem dos executores. Registrar diferenças inerentes aos runtimes, incluindo prompts internos e políticas de erro que não puderem ser iguados.

17 Métricas logs e avaliação de resultados

Métrica	Como observar
Sucesso da tarefa	Todos os requisitos atendidos, evidências válidas e ações confirmadas.
Uso de ferramentas	Seleção, argumentos e operações coerentes, sem escritas indevidas.
Qualidade factual	Afirmações sustentadas pelas fontes e limites corretamente declarados.
Latência	Tempo total por tarefa e por etapa; mediana e percentis definidos no protocolo.
Custo	Consumo medido e parcela de infraestrutura conforme critério documentado.
Consistência	Variação entre repetições da mesma tarefa.
Recuperação de falhas	Comportamento diante de erros controlados, se incluídos no protocolo.

Proposta para o artigo: sucesso da tarefa como métrica primária e uso de ferramentas como secundária. Isso altera o rascunho anterior e precisa ser registrado antes da coleta. Aceitar caminhos alternativos válidos; exigir uma sequência única pode penalizar um agente correto.

Exemplo de evento registrado

```
{
  "run_id": "RUN_001",
  "task_id": "TASK_014",
  "executor": "langgraph",
  "base_version": "piloto_v1",
  "evento": "tool_end",
  "tool_call_id": "CALL_003",
  "ferramenta": "registrar_pendencia",
  "duracao_ms": 185,
  "status": "ok",
  "artefato_id": "PEND_004"
}
```

O evento completo também identifica modelo, commit, horários e referências aos argumentos e retornos. Dados sensíveis devem ter acesso restrito; evitar duplicá-los em logs públicos. Contagem de tokens indisponível deve ser registrada como ausente, não como zero.

A conferência de escrita usa banco e artefatos. Para respostas textuais, aplicar rubrica, ocultar o executor e usar dois avaliadores em uma amostra definida previamente, resolvendo divergências. Um LLM avaliador pode auxiliar, mas precisa de validação e não deve ser a única evidência de sucesso.

Apresentar diferenças com incerteza e análise de erros. Custo por tarefa concluída é custo total dividido pelo número de sucessos; se não houver sucesso, registrar custo por sucesso como indefinido ou infinito, explicitando a convenção.

18 Cronograma até dezembro

Calendário proposto para 15 pessoas, incluindo a coordenação, com hipótese inicial de 4–6 horas semanais por pessoa. Confirmar disponibilidade na abertura e ajustar tarefas à experiência. A escrita começa junto com a definição do estudo.

Período de 2026	Entrega técnica	Pesquisa e artigo
21–27 setembro	Ambiente, equipes e exercício inicial.	Pergunta e levantamento de veículos.
28 set a 4 out	API de consulta e interfaces combinadas.	Tarefas e critérios iniciais.
5–11 outubro	Base inicial e ferramenta em cada agente.	Casos de desenvolvimento e literatura.
12–18 outubro	Consulta e escrita com documentos preparados.	Validação dos casos com especialista.
19–25 outubro	Integração, logs, erros e persistência.	Descrição da base e arquitetura.
26 out a 1 nov	Piloto; fim de funcionalidades em 30/10.	Ajuste final das rubricas.
2–8 novembro	Congelamento de versões até 06/11.	Protocolo e teste final fechados.
9–15 novembro	Rodadas do experimento principal.	Avaliação e conferência dos registros.
16–22 novembro	Reserva para falhas de execução documentadas.	Resultados fechados até 20/11.
23–29 novembro	Documentação para reprodução.	Artigo completo em 27/11.
30 nov a 6 dez	Verificação dos materiais.	Revisão do orientador e formatação.
7–10 dezembro	Demonstração e entrega.	Manuscrito pronto para submissão.

Uma correção que altere o comportamento após o congelamento exige nova versão e regra de reexecução comparável. Não escolher silenciosamente apenas as rodadas favoráveis. O prazo de submissão efetiva depende do calendário do veículo.

Se o projeto atrasar, reduzir interface, segundo modelo e recursos extras de busca. Preservar a avaliação e a revisão do artigo. Os marcos de 30/10, 06/11 e 27/11 servem para explicitar o impacto de mudanças de escopo.

19 Formação e primeiras entregas por equipe

Todos estudam o básico de Python, Git, HTTP, JSON e funcionamento de LLMs com ferramentas. Depois, cada frente aprofunda as tecnologias ligadas à sua entrega. O estudo deve resultar em uma demonstração verificável.

Frente	Estudar	Primeira entrega
Documentos	PDF, OCR, metadados, chunks, busca e embeddings.	Extrair cinco processos fictícios e conferir fontes de dez consultas.
Backend	FastAPI, Pydantic, SQL, transações e testes.	Consulta de processo e pendência com validação.
Hermes	Repositório adotado, configuração e extensão de ferramentas.	Chamar ferramenta fictícia e exportar o histórico.
LangGraph	Estado, nós, mensagens e arestas condicionais.	Ciclo modelo–ferramenta com encerramento.
Infraestrutura	Linux, SSH, Docker, redes, volumes e backups.	Subir API e banco e demonstrar persistência após reinício.
Metodologia	Tarefas, rubricas, métricas e desenho pareado.	Cinco tarefas com critérios e referências validadas.

Ritmo de acompanhamento

- Segunda: 30 minutos com os responsáveis para definir entregas e resolver dependências.
- Durante a semana: registrar bloqueios e manter tarefas pequenas, com responsável e revisor.
- Sexta: demonstração de até 45 minutos com o grupo, mostrando entradas, execução e evidências.
- Primeiro mês: uma sessão técnica semanal ligada ao próximo marco.

Critério comum de conclusão

Uma tarefa só está pronta quando alguém consegue reproduzir o comportamento esperado, consultar os registros e explicar a limitação conhecida. Código integrado deve incluir uma verificação pertinente: erro de referência, persistência, isolamento ou idempotência, por exemplo. A intenção é verificar riscos reais, não gerar testes que apenas repetem a implementação.

A revisão cruzada entre Hermes e LangGraph ajuda a identificar configurações desiguais. A equipe de avaliação preserva os casos reservados e coordena a conferência independente dos resultados.

20 Artigo entregáveis e decisões pendentes

Estrutura de escrita proposta

Parte do artigo	Responsável principal	Contribuição
Introdução e objetivo	Coordenação e metodologia	Problema, pergunta e recorte do estudo.
Trabalhos relacionados	Metodologia com apoio de todos	Fontes lidas e comparação com estudos anteriores.
Corpus e tarefas	Documentos e metodologia	Origem, cobertura, preparação e separação de casos.
Implementação	Backend, Hermes e LangGraph	Ferramentas, fluxos e configurações reproduzíveis.
Ambiente experimental	Infraestrutura	Recursos, versões e procedimento de execução.
Resultados e discussão	Metodologia e coordenação	Métricas, incerteza, falhas e interpretação.
Limitações e conclusão	Coordenação com revisão coletiva	O que os resultados sustentam e o que permanece aberto.

Pacote final

Entregar manuscrito, código do projeto, configurações sem segredos, catálogo de ferramentas, protocolo, resultados agregados e instruções de reprodução. A possibilidade de compartilhar documentos e logs brutos depende das restrições do corpus; preparar exemplos sintéticos ou dados liberados quando necessário. Código aberto, por si só, não garante ausência de viés.

Decisões para a primeira reunião

- Confirmar pessoas, disponibilidade e responsáveis por frente.
- Identificar o Hermes exato e o responsável por orientar essa integração.
- Escolher procedimento administrativo e pessoa que validará seus requisitos.
- Definir corpus acessível, cobertura, tratamento de dados e endpoint permitido.
- Escolher um modelo inicial, limites de uso e infraestrutura do piloto.
- Selecionar veículo do artigo e combinar acompanhamento com o orientador.

A comparação deve ser registrada como estudo das implementações avaliadas. Uma diferença pequena, um empate ou falhas recorrentes são resultados relevantes quando o protocolo permite explicá-los. O objetivo da equipe é produzir evidências úteis e reproduzíveis.

21 Glossário para a equipe

Termo	Significado neste projeto
API	Interface que permite solicitar operações ao backend.
Backend	Código que implementa consultas, validações e gravações.
Tool	Função com entrada e saída definidas que o agente pode solicitar.
Tool calling	Produção, pelo modelo, de uma solicitação estruturada de ferramenta.
LLM	Modelo de linguagem utilizado para interpretar e gerar conteúdo.
Orquestração	Controle da sequência entre modelo, ferramentas, estado e encerramento.
Adaptador	Camada que traduz uma interface para a interface comum do projeto.
Chunk	Trecho recuperável de um documento, com sua localização.
Embedding	Vetor numérico usado para comparar similaridade de conteúdo.
RAG	Recuperação de conteúdo para fundamentar a geração do modelo.
OCR	Reconhecimento de texto em imagens ou páginas digitalizadas.
Worker	Processo que executa tarefas pendentes fora da requisição inicial.
VPS	Servidor virtual que hospeda os serviços da aplicação.
Container	Ambiente de execução de um serviço com suas dependências.
Volume	Armazenamento persistente conectado a um container.
Idempotência	Repetir uma operação lógica sem duplicar seu efeito.
Trace	Histórico que conecta os eventos de uma execução.
Checkpoint	Estado salvo para permitir retomada de uma execução.
Benchmark	Conjunto de tarefas e regras para comparar sistemas.
Gabarito	Critérios e evidências usados para julgar a tarefa.

22 Referências e notas dos exemplos

Fontes consultadas em 17 de setembro de 2026. Os links apoiam as definições e os mecanismos técnicos; a arquitetura, a divisão de equipes e o cronograma são propostas específicas para este projeto.

[1 TRF4 — Origem e uso do SEI](#)

[2 UFMG — Sistema Eletrônico de Informações](#)

[3 PEN — Manuais do SEI](#)

[4 FastAPI — Corpos de requisição e modelos](#)

[5 PyMuPDF — Extração de texto](#)

[6 pgvector — Busca vetorial no PostgreSQL](#)

[7 LangGraph — Visão geral](#)

[8 LangGraph — Quickstart de agente com ferramentas](#)

[9 LangGraph — Persistência](#)

[10 Hostinger — Template Docker em VPS](#)

[11 Docker — Compose](#)

[12 Dror et al. 2018 — Statistical Significance in NLP](#)

Documento de referência do projeto

[hermes_bench_metodologia.html](#), rascunho datado de 8 de setembro de 2026, fornecido pela coordenação. O guia incorpora o recorte discutido posteriormente: documentos do SEI coletados manualmente e comparação inicial entre dois executores. As alterações de escopo e métrica devem ser incorporadas ao protocolo antes das rodadas finais.

Limites dos exemplos técnicos

A extração é um exemplo local. A API é didática e usa armazenamento em memória. O cliente HTTP e o grafo são esqueletos de integração que dependem de serviços e modelo configurados. O trecho Hermes é pseudocódigo. O Compose exige um backend preparado. Nenhum desses trechos representa um sistema de produção pronto.